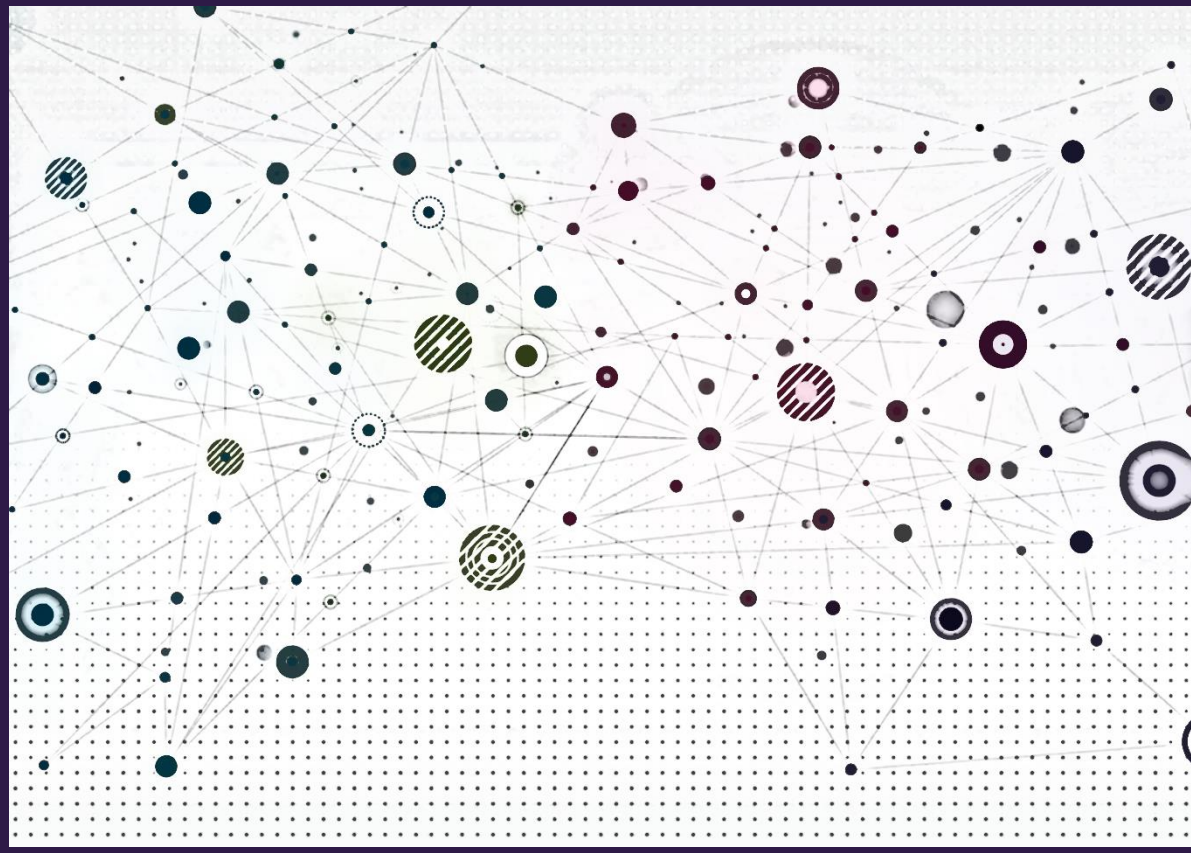
School of COMPUTING
Clemson University

Shallow Node Embeddings



Overview

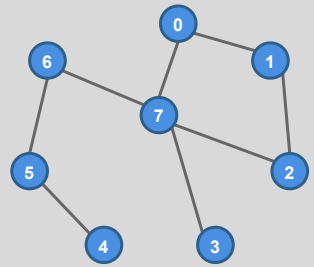
- Motivation
- Shallow node embedding framework
- Node embedding methods
  - Factorization approaches
  - Random walk approaches
- Graph embedding
- Limitations of matrix factorization and random walk embeddings

Motivation

Previous lab – Machine learning with engineered features

- In a standard machine learning approach, we extract mathematically formulated metrics from an input graph to train and make inferences with a statistical learning model
- As with many other areas (e.g., computer vision, natural language processing), it is frequently beneficial to automate feature learning and bypass feature engineering

Input graph



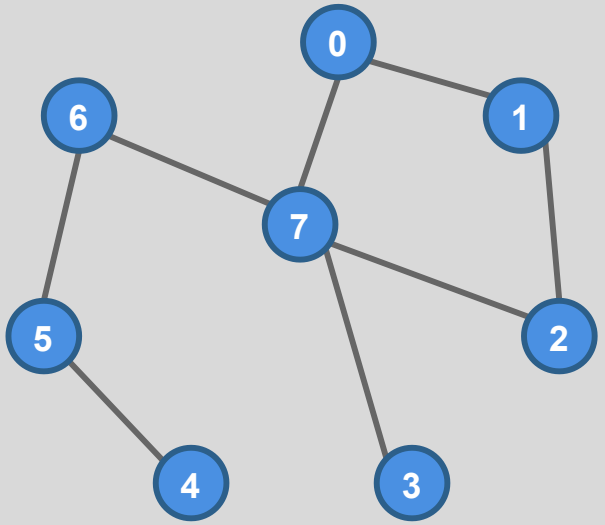
Engineered Features (centrality, cliques, k-cores, diameter, etc)

Machine learning algorithm

Inference

Latent representation learning for graphs

We would like to learn node, edge, and graph representations (embeddings) that are useful for many tasks



$f: u \rightarrow \mathbb{R}^d$

Vector in $\mathbb{R}^d$

- f is a function that maps an input, u, (node, edge, or graph) to numerical vector in $\mathbb{R}^d$
- In this course, f will typically be a neural network

Why learn embeddings?

- Eliminates reliance on handcrafted features
- May capture network information that is not in engineered features
- Potentially useful for many downstream tasks
- Pretraining with self-supervised learning can reduce labeled data requirements for downstream tasks

Embedding in $\mathbb{R}^d$

Pre-trained node or graph via self-supervised learning

Downstream Tasks

- Node classification
- Graph classification
- Clustering
- Graph generation
- … and so on

Example node embeddings

Image credit: Perozzi 2014

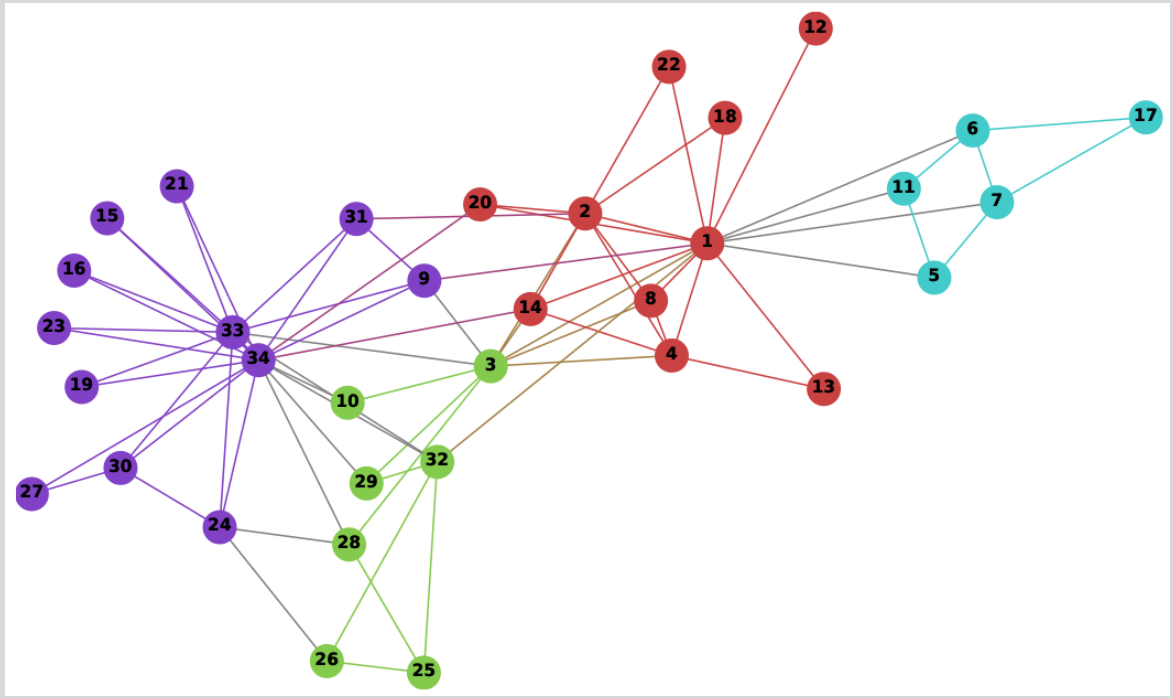
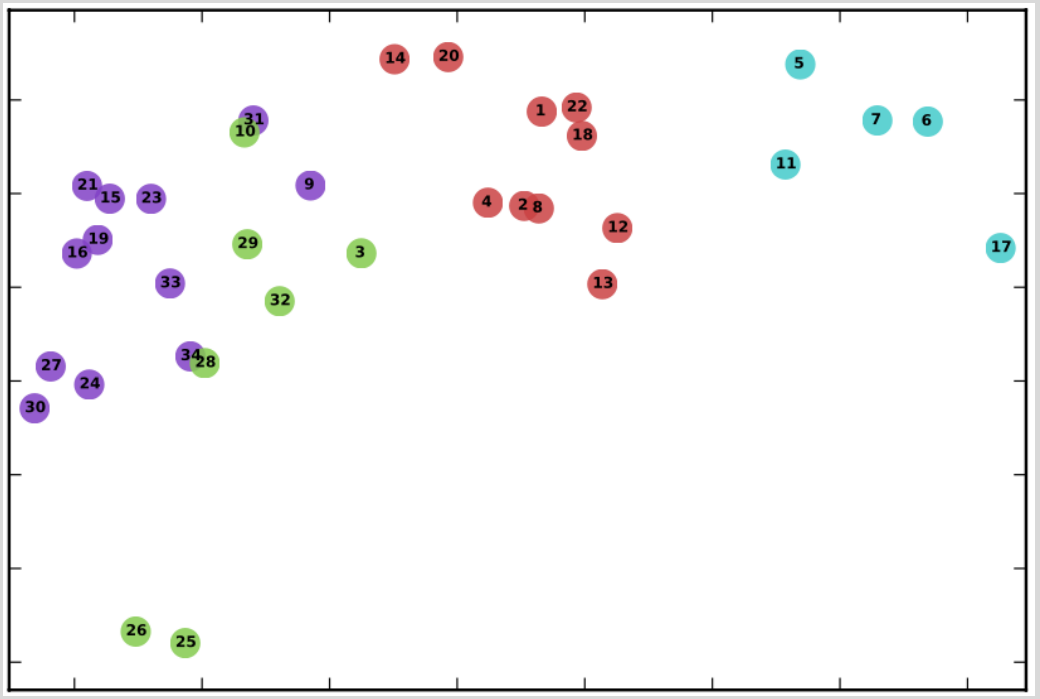


Image credit: Perozzi 2014



Embeddings in $\mathbb{R}^2$ for Zachery's Karate network. Notice how community structure of the graph is maintained by the embeddings.

Shallow Node Embedding Framework

Encoder-decoder conceptual description

- We will adopt an encoder-decoder paradigm where:
  - Encoder maps each node (or edge) to an embedding, i.e., a vector in $\mathbb{R}^d$
  - Decoder takes one or more embeddings and infers information about a node, edge, or graph.
  - Typically, we will only care about the encoder for downstream tasks, but both are necessary to form the embeddings

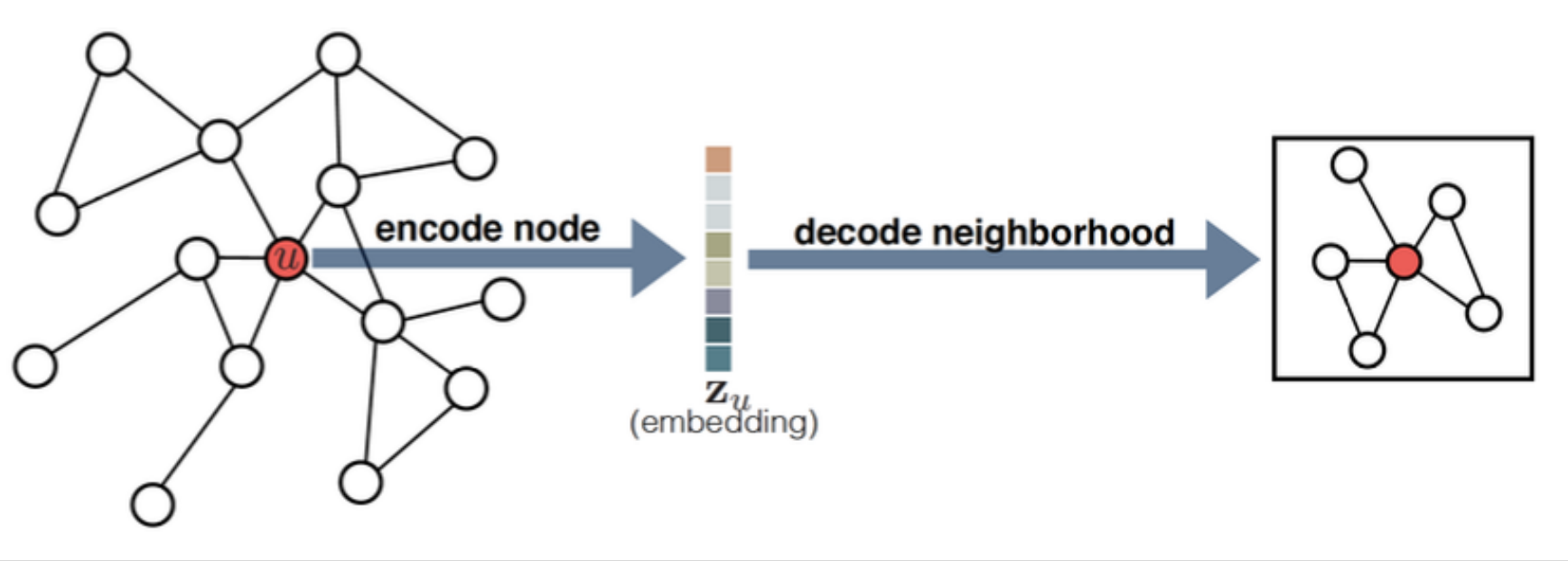


Image credit: Hamilton 2020

Encoder-decoder components

- The Encoder, ENC, is a function (e.g., a neural network) that maps nodes to an $\mathbb{R}^d$ embedding
- A node (or edge) similarity measure, $\mathbf{S}[u, v]$, that we'll use to assess performance of the encoder-decoder model
- The Decoder, DEC, infers the similarity measure from embedding inputs
- Optimization approach to learn model parameters so that:

$$DEC(\boldsymbol{e}_u, \boldsymbol{e}_v) \approx \mathbf{S}[u, v] \text{ for all nodes (or edges) in the network}$$

$$DEC\big(ENC(u), ENC(v)\big) \approx \mathbf{S}[u, v] \text{ for all nodes (or edges) in the network}$$

Encoder concept in detail

- The *encoder*, $ENC(v) = \mathbf{z}_v$, is any function (e.g., a neural network) that maps a node, v, to a vector, $\mathbf{z}_v \in \mathbb{R}^d$
- d is often called the embedding dimension or size
- Today, we will focus on shallow methods that only encode network structure
- Later, we'll see graph neural networks that also use node features

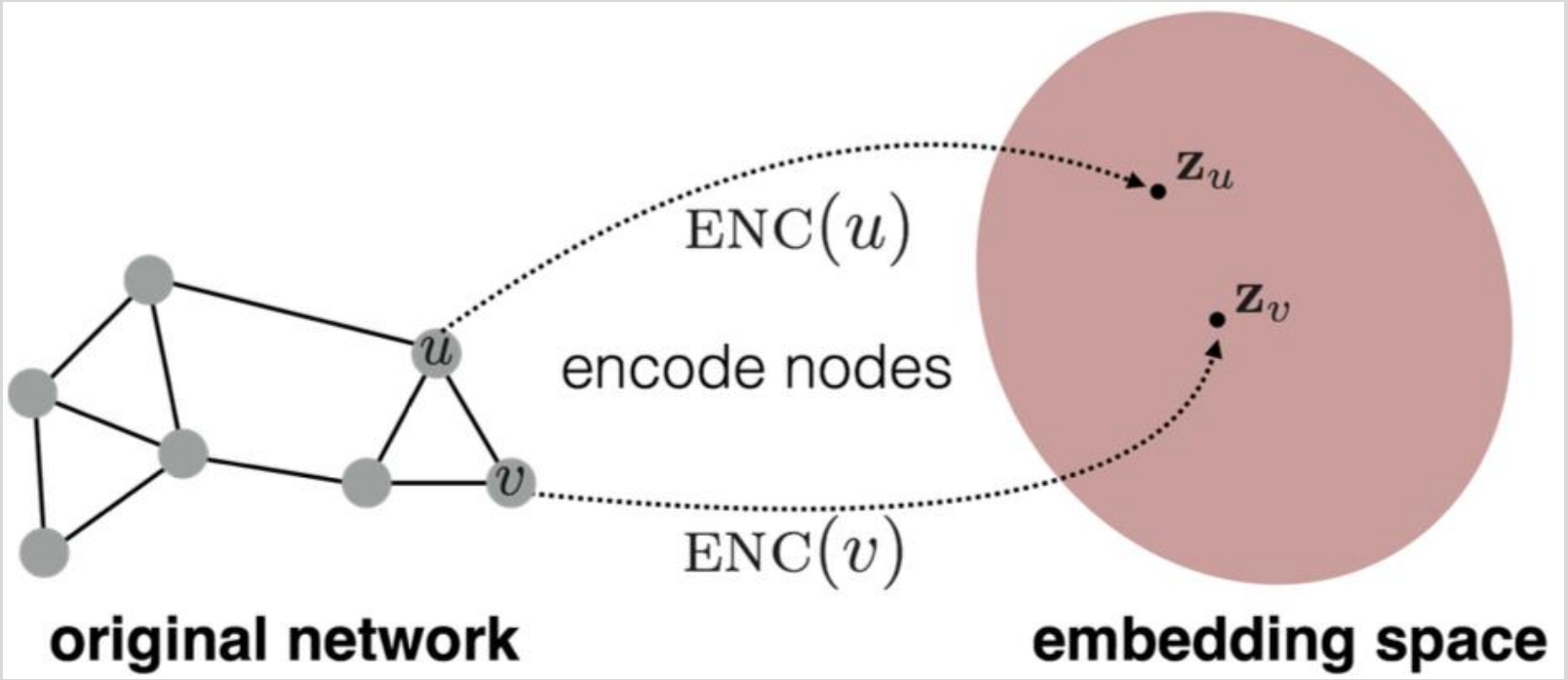
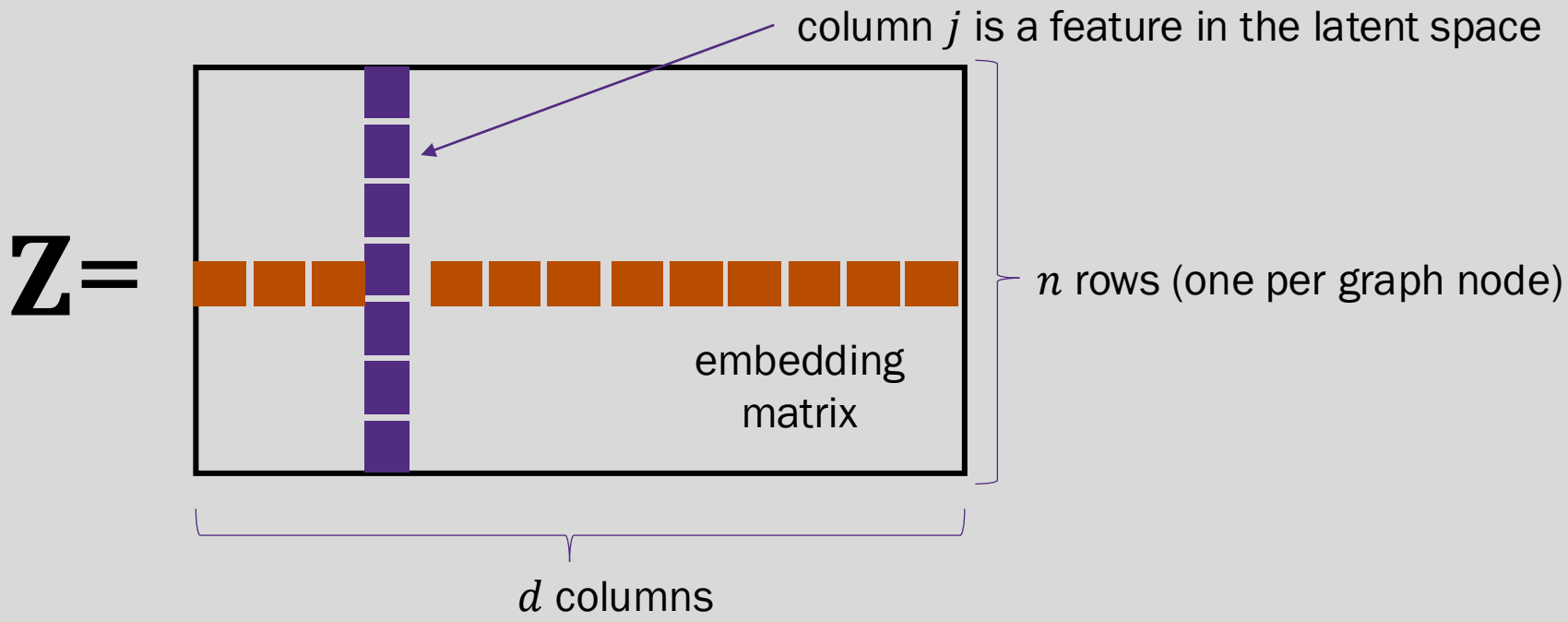


Image credit: Hamilton 2020

Encoder as a lookup function

- In the simplest case:
  - The graph is static (no nodes added or removed)
  - The embedding is constant for every node and stored in an embedding matrix, $\mathbf{Z} \in \mathbb{R}^{n \times d}$ where d is the embedding dimension and n is the number of nodes
  - The encoder is just a lookup, i.e., given a node ID, i, it returns the i^{th} row from $\mathbf{Z}$



Embedding learning through loss optimization

- We consider a **pairwise decoder**, $DEC(\mathbf{z}_u, \mathbf{z}_v)$, that is a function of any two node embedding inputs
- Let $\mathbf{S}[u, v] \in \mathbb{R}^+$ be a similarity measure of nodes u and v
- Our goal is to construct an encoder-decoder model such that

$$DEC(\mathbf{z}_u, \mathbf{z}_v) \approx \mathbf{S}[u, v]$$

- Standard approach is to minimize the empirical loss, $\mathcal{L}$, over a set of training node pairs $\mathcal{D}$:

$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} l\big[DEC(\mathbf{z}_u, \mathbf{z}_v), \mathbf{S}[u, v]\big]$$

  where $l \in \mathbb{R} \times \mathbb{R} \longrightarrow \mathbb{R}$ is a loss function
- We will discuss specific similarity measures and loss functions in the following slides

Shallow node embedding qualities

- Unsupervised or self-supervised learning task (depends on specific method)
- Node features are not used
- Node labels are not used
- Goal is to learn latent features that the decoder can use to reconstruct a similarity measure derived from network structure
- The node embeddings are intended to be useful for any task where network structure is informative

School of COMPUTING
Clemson University

Node Embedding with Matrix Factorization

Matrix factorization embeddings

- Simplest node similarity: Nodes u and v are similar if they are connected by an edge
- Let the decoder be $DEC(\mathbf{z}_u, \mathbf{z}_v) = \mathbf{z}_v^T \mathbf{z}_u$ (inner product decoder)
- This means we want : $\mathbf{z}_v^T \mathbf{z}_u = a_{u,v}$ which is the (u, v) entry of the graph adjacency matrix $\mathbf{A}$
- Equivalently, letting $\mathbf{Z} \in \mathbb{R}^{n \times d}$ be the embedding matrix, we want $\mathbf{Z}\mathbf{Z}^T = \mathbf{A}$

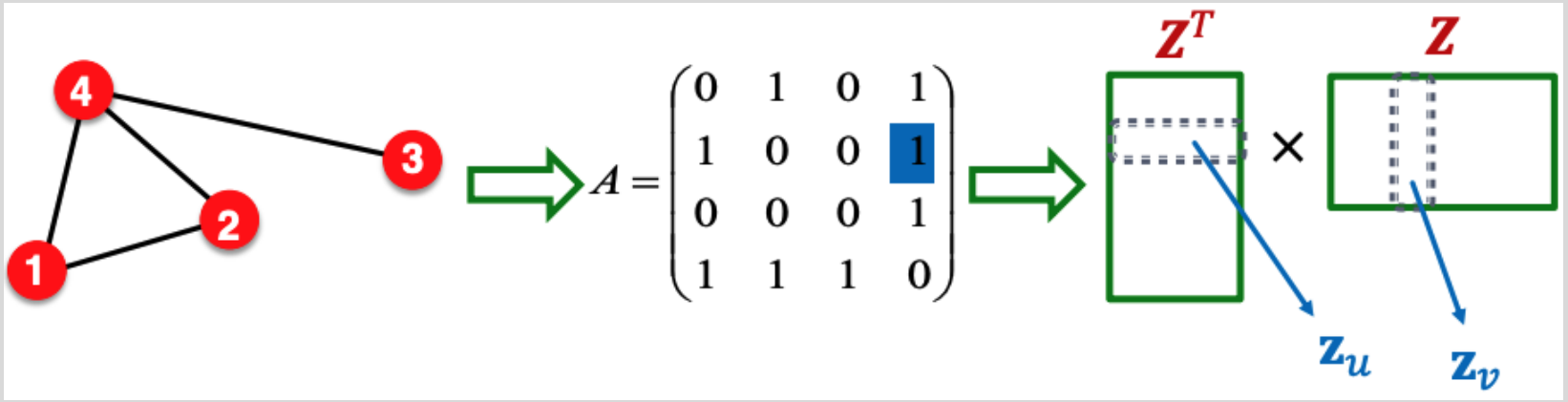


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Matrix factorization embeddings

- The embedding dimension d (number of columns in $\mathbf{Z}$) is much smaller than number of nodes n
- Exact factorization $\mathbf{Z}\mathbf{Z}^T = \mathbf{A}$ is generally not possible
- However, we can learn $\mathbf{Z}$ approximately
- Objective function: $\min_{\mathbf{Z}} \|\mathbf{A} - \mathbf{Z}\mathbf{Z}^T\|_2$ - here the similarity function is the adjacency matrix, $\boldsymbol{S} = \mathbf{A}$
  - We optimize $\mathbf{Z}$ such that it minimizes the L2 norm (Frobenius norm)

$$\|\mathbf{A} - \mathbf{Z}\mathbf{Z}^T\|_2 = \sqrt{\sum_{i=1}^{n}\sum_{j=1}^{n}\left(a_{ij} - \left(\mathbf{z}_i^T\mathbf{z}_j\right)\right)^2}$$

- Inner product decoder with node similarity defined by edge connectivity is equivalent to matrix factorization of $\mathbf{A}$

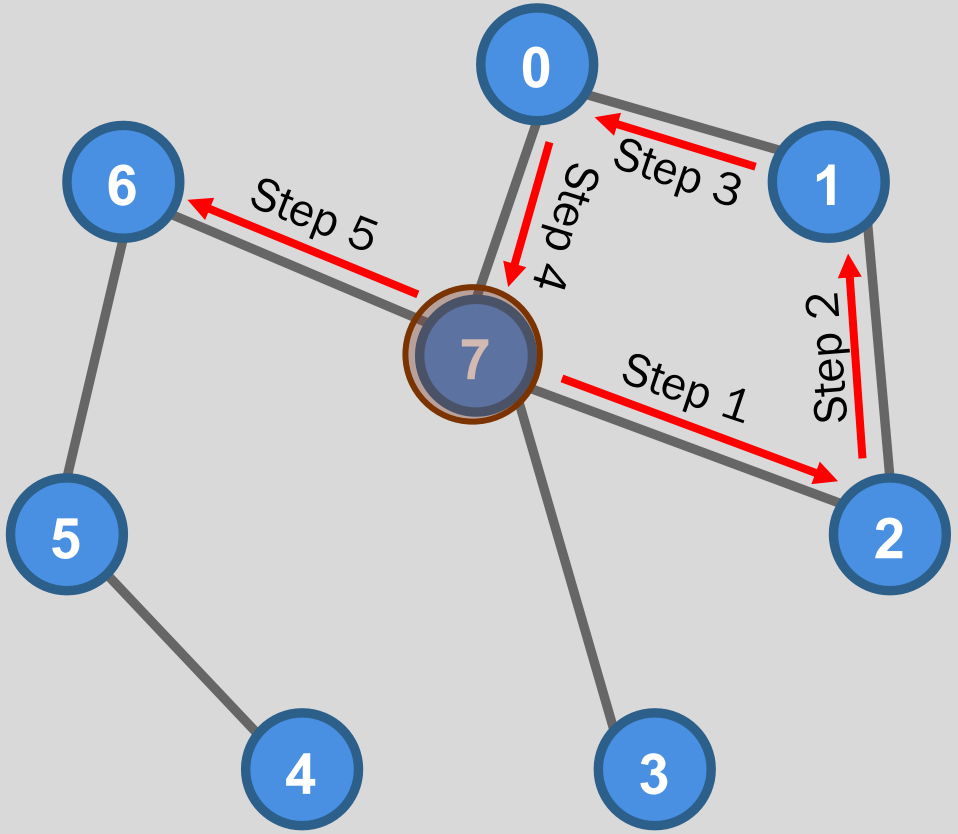
Matrix factorization embeddings

- Other methods adopt the same approach for matrix factorization embeddings
  - Same inner product decoder $DEC(\mathbf{z}_u, \mathbf{z}_v) = \mathbf{z}_v^T \mathbf{z}_u$
  - Same objective function $\min_{\mathbf{Z}} \|\mathbf{S} - \mathbf{Z}\mathbf{Z}^T\|_2$
- The difference is how the similarity, $\mathbf{S}$, is defined (there are many alternatives)

School of COMPUTING
Clemson University

Node Embedding with Random Walks

Random walk



Starting at node 7, and taking 5 random steps yields the walk:

$$7 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 7 \rightarrow 6$$

1. Start at a randomly selected node, u. Set $N = n(u)$ to the neighbors of u
2. Randomly pick a node v from N.
3. Set $u = v$
4. Set $N = n(u)$
5. Repeat 2-4 for $T - 1$ times where T is the desired path length (edge count)

- Loops are allowed, including going back to the previous node from the current node

Random walk embedding concept

- Intuition: Two nodes that co-occur on short random walks are likely to be structurally similar (share neighbors) and should have similar embeddings
- We can express this intuition mathematically by setting the similarity function, $\mathbf{S}[u, v]$, to

$$\mathbf{S}[u, v] \equiv p_{G,T}(v|u)$$

where $p_{G,T}(v|u)$ is the probability of visiting node v on a length-T random walk starting at node u on graph G

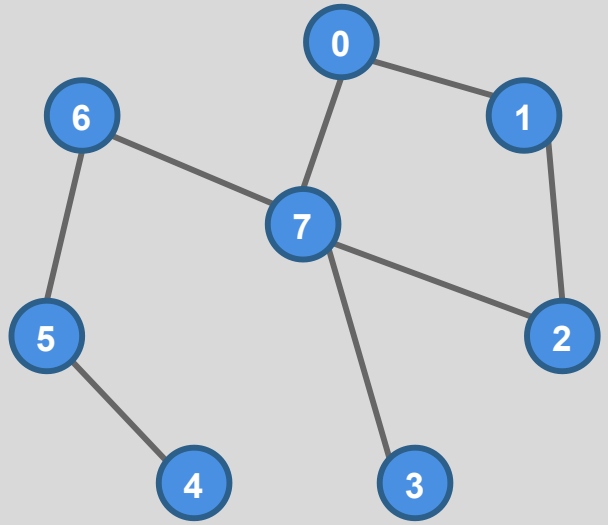
- Our goal is to develop an encoder-decoder model such that

$$DEC(\mathbf{z}_u, \mathbf{z}_v) \approx p_{G,T}(v|u)$$

for any pair of nodes u and v with embeddings $\mathbf{z}_u$ and $\mathbf{z}_v$

Random walk similarity measure

- In the previous slide, we set $\mathbf{S}[u, v] \equiv p_{G,T}(v|u)$
- But we don't know $p_{G,T}(v|u)$. What can we do?
- Generate many length-T random walks using a walk strategy R
- Estimate probabilities empirically from the random walks



Length-3 Random Walks
7→2→1
7→0→1
7→3→7
7→6→5
7→0→1

Probability Estimates

$p_{G,3}(0|7) = {}^{2}/_{5}$

$p_{G,3}(1|7) = {}^{3}/_{5}$

$p_{G,3}(2|7) = {}^{1}/_{5}$

$p_{G,3}(3|7) = {}^{1}/_{5}$

$p_{G,3}(4|7) = {}^{0}/_{5}$

$p_{G,3}(5|7) = {}^{1}/_{5}$

$p_{G,3}(6|7) = {}^{1}/_{5}$

Random walk decoder

- A natural choice for a decoder to approximate $p_{G,T}(v|u)$ is the softmax

$$DEC(\mathbf{z}_u, \mathbf{z}_v) = \frac{exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)}$$

  where V is the set of all nodes in the graph

- This will result in all values of $DEC(\mathbf{z}_u, \mathbf{z}_v)$ being in $[0,1]$ and $\sum_{v \in V} DEC(\mathbf{z}_u, \mathbf{z}_v) = 1$ which can be interpreted as a probability distribution
- We want to optimize the encoder-decoder, that is we want to learn embeddings such that

$$DEC(\mathbf{z}_u, \mathbf{z}_v) = \frac{exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)} \approx p_{G,T}(v|u)$$

Naïve random walk embedding optimization

- We could attempt to minimize the square difference

$$\left(\frac{exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)} - \hat{p}_{G,T}(v|u) \right)^2$$

where $\hat{p}_{G,T}(v|u)$ is an empirical estimate of $p_{G,T}(v|u)$ based on random walk samples.

  - Intractable for large graphs as requires counting co-occurrences for every pair of nodes which is $O(n^2)$
  - In most networks, most pairs will never co-occur on short walks leading to 0 empirical probability which could yield large negative embeddings
- What else can we try?

Random walk embedding optimization by maximum likelihood

- Let's consider a given node u and the set of all nodes $N_R(u)$ visited on some random walk starting at u using a random walk policy R
- Applying maximum likelihood theory, we seek to minimize the objective function

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -log \left[\frac{exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)} \right]$$

Sum over all nodes u

Sum over nodes v seen on random walks starting from u

Estimated probability of u and v co-occurring on a random walk

- Optimization means finding embeddings $\mathbf{z}$ that minimize $\mathcal{L}$

Random walk embedding optimization by maximum likelihood

- Why does this work?

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -log\left[\frac{exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)}\right]$$

$$= \sum_{u \in V} \sum_{v \in N_R(u)} -log[exp(\mathbf{z}_u^T \mathbf{z}_v)] + log\left[\sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)\right]$$

- Minimizing this function requires
  - Pushing $\uparrow \mathbf{z}_u^T \mathbf{z}_v$ - requires $\mathbf{z}_u$ and $\mathbf{z}_v$ to be similar (ideally, correlated)
  - Pushing $\downarrow \sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)$ - requires $\mathbf{z}_u$ to be dissimilar (ideally, anticorrelated) from all $\mathbf{z}_n$ especially $\mathbf{z}_n \notin N_R(u)$
- Eliminates the need to count co-occurrence for every pair of nodes and addresses 0 empirical probability issues, but …

Random walk embedding optimization by maximum likelihood

- Too computationally expensive

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -log \left[\frac{exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} exp(\mathbf{z}_u^T \mathbf{z}_n)} \right]$$

Nested summation over all nodes in the graph gives $O(n^2)$ complexity. This is prohibitive on large graphs.

Negative sampling

Revise our loss function using the approximation*

$$-log\left[\frac{exp(\boldsymbol{z}_u^T\boldsymbol{z}_v)}{\sum_{n\in V} exp(\boldsymbol{z}_u^T\boldsymbol{z}_n)}\right]$$

$$\approx -\log[\sigma(\boldsymbol{z}_u^T\boldsymbol{z}_v)] - \sum_{i=1}^{k} \log\left[\sigma\left(-\boldsymbol{z}_u^T\boldsymbol{z}_{n_i}\right)\right], n_i \sim P_V$$

random distribution over all nodes

sigmoid function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

- Normalize against k randomly selected *negative samples*
- A form of *Noise Contrastive Estimation (NCE)*
- Eliminates the need to compute the denominator over all nodes
- Complexity is independent of number of nodes in network

*See https://arxiv.org/pdf/1402.3722

Negative sampling

Revise our loss function using the approximation*

$$-log\left[\frac{exp(\boldsymbol{z}_u^T\boldsymbol{z}_v)}{\sum_{n\in V} exp(\boldsymbol{z}_u^T\boldsymbol{z}_n)}\right]$$

$$\approx -\log[\sigma(\boldsymbol{z}_u^T\boldsymbol{z}_v)] - \sum_{i=1}^{k} \log\left[\sigma\left(-\boldsymbol{z}_u^T\boldsymbol{z}_{n_i}\right)\right], n_i \sim P_V$$

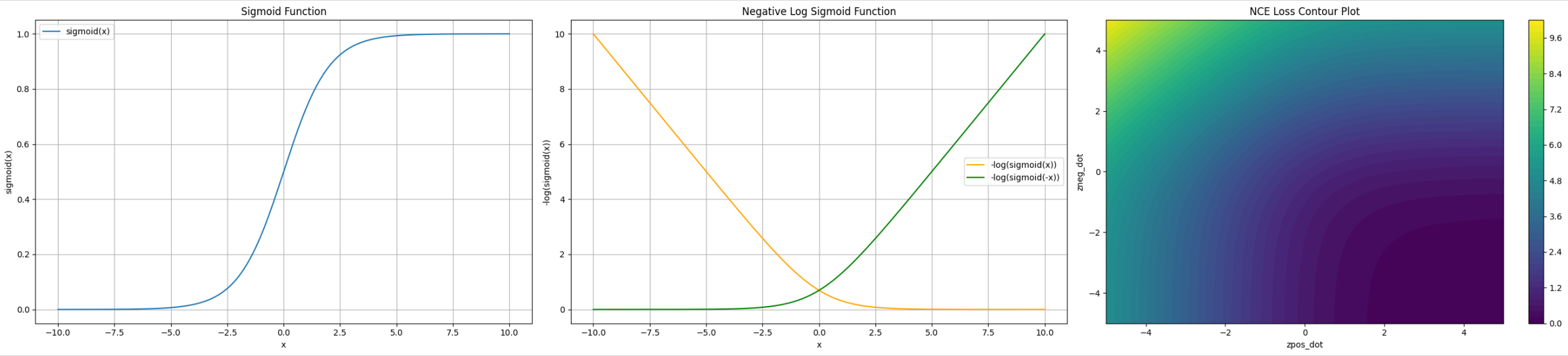
- Sample k negative nodes n_i with probability proportional to node degree
- Higher k gives more robust estimates but also higher bias on negative events (meaning, the negative sample node might actually occur on random walks we just didn't take enough samples)
- In practice, $k \in [5, 20]$

*See https://arxiv.org/pdf/1402.3722

NCE (loss function version)

$$-\left[\log[\sigma(\mathbf{z}_u^T\mathbf{z}_v)] + \sum_{i=1}^{k} \log[\sigma(-\mathbf{z}_u^T\mathbf{z}_{n_i})], n_i \sim P_V\right]$$

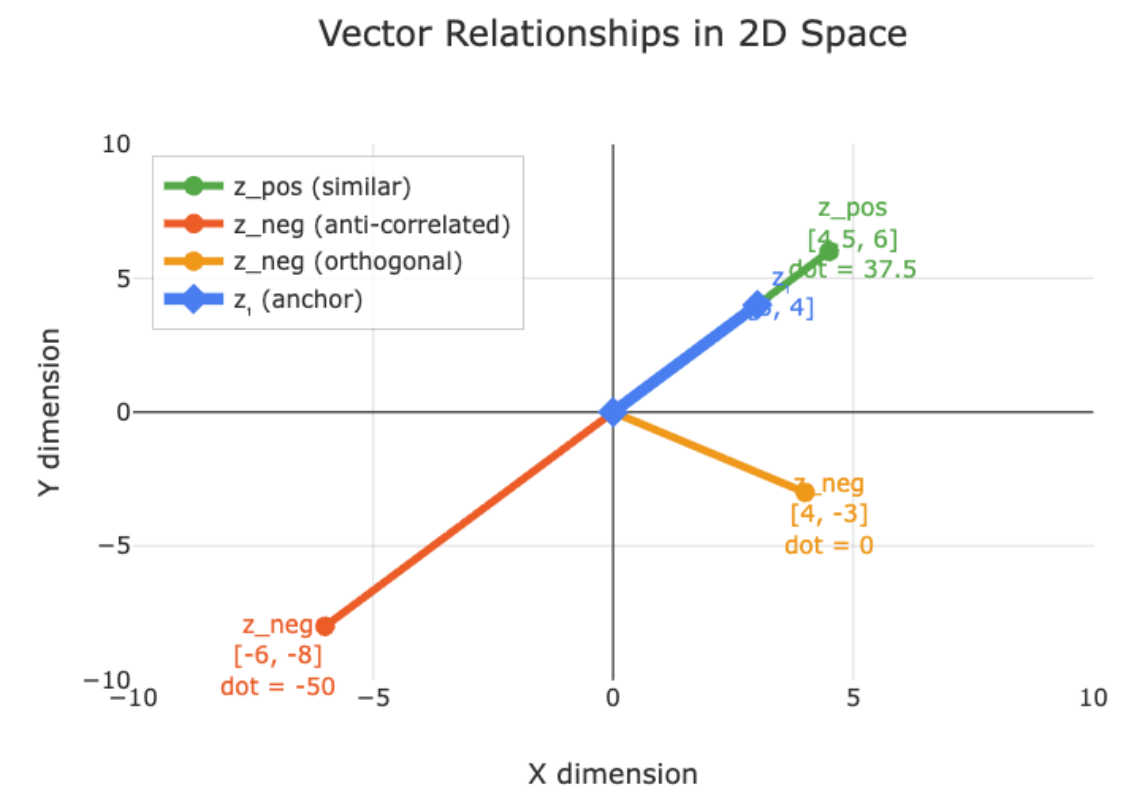
Contour plot of the NCE loss function. The x-axis is the value of $\mathbf{z}_u^T\mathbf{z}_v$. The y-axis is the value of $\mathbf{z}_u^T\mathbf{z}_{n_i}$.



The orange curve represents the value of $-\log[\sigma(\mathbf{x})]$ where $\mathbf{x} = \mathbf{z}_u^T\mathbf{z}_v$. The best we can do is make this dot product large. The green curve represents $-\log[\sigma(-\mathbf{x})]$ where $\mathbf{x} = \mathbf{z}_u^T\mathbf{z}_{n_i}$. The best we can do is make this value negative with large magnitude.

Understanding NCE loss

$$-\left[\log[\sigma(\mathbf{z}_u^T\mathbf{z}_v)] + \sum_{i=1}^{k} \log[\sigma(-\mathbf{z}_u^T\mathbf{z}_{n_i})], n_i \sim P_V\right]$$



Key Insight: Both similar vectors (positive pairs) and anti-correlated vectors (negative pairs) achieve low loss values. The model learns both *attraction* and *repulsion* relationships!

A. Similar Vectors (Positive Pair)

`z1 = [3, 4], z_pos = [4.5, 6]`

`dot_product = 3×4.5 + 4×6 =` **37.5**

`sigmoid(37.5) =` **≈ 1.000**

`log(sigmoid(37.5)) =` **≈ 0.000**

✓ *Low loss for positive pair - vectors learn to align*

B. Anti-correlated Vectors (Negative Pair)

`z1 = [3, 4], z_neg = [-6, -8]`

`dot_product = 3×(-6) + 4×(-8) =` **-50**

`sigmoid(-(-50)) = sigmoid(50) =` **≈ 1.000**

`log(sigmoid(50)) =` **≈ 0.000**

✓ *Low loss for negative pair - strong opposition is good!*

C. Orthogonal Vectors (Negative Pair)

`z1 = [3, 4], z_neg = [4, -3]`

`dot_product = 3×4 + 4×(-3) =` **0**

`sigmoid(-0) = sigmoid(0) =` **0.500**

`log(sigmoid(0)) =` **≈ -0.693**

↑ *Higher loss - neutral relationship, room for improvement*

How do we optimize our objective function?

- Assume our loss function is $\mathcal{L}$ and we want to optimize the embedding $\mathbf{z}_u$
- Stochastic Gradient Descent – process one sample at a time
  1. Randomly initialize $\mathbf{z}_m$ for all nodes m
  2. For each random walk sample $s = \{u, v_1, \cdots, v_w, n_1, \cdots, n_k\}$
     a. Compute the partial derivatives $\frac{\delta \mathcal{L}}{\delta \mathbf{z}_m}$ for all nodes $m \in s$
     b. For all nodes m, step in the negative gradient direction $z_m \leftarrow z_m - \alpha \frac{\delta \mathcal{L}}{\delta \mathbf{z}_m}$ where α is the learning rate
  3. Repeat 2 until convergence
- Completion of step 2 represents one epoch

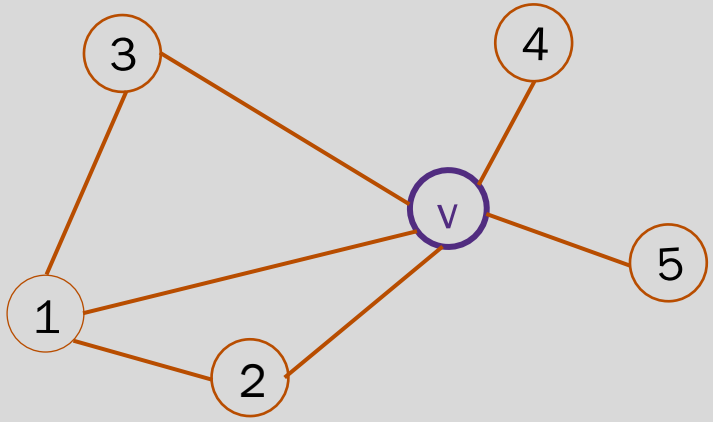
Random walk summary

1. Generate short fixed-length random walks starting from each node, u, on the graph using a random walk policy R
2. For each node u, determine the neighborhood $N_R(u) = \{v_1, \cdots, v_w\}$ from the walks and combine with k randomly selected negative nodes $\{n_1, \cdots, n_k\}$. This represents a single sample $s = \{u, v_1, \cdots, v_w, n_1, \cdots, n_k\}$
3. Optimize the embeddings $\mathbf{Z}$ using stochastic gradient descent with the NCE loss function (using negative sampling)

$$\mathcal{L}_s = \sum_{v \in N_R(u)} -log[\sigma(\mathbf{z}_u^T \mathbf{z}_v)] - \sum_{i=1}^{k} \log[\sigma(-\mathbf{z}_u^T \mathbf{z}_{n_i})]$$

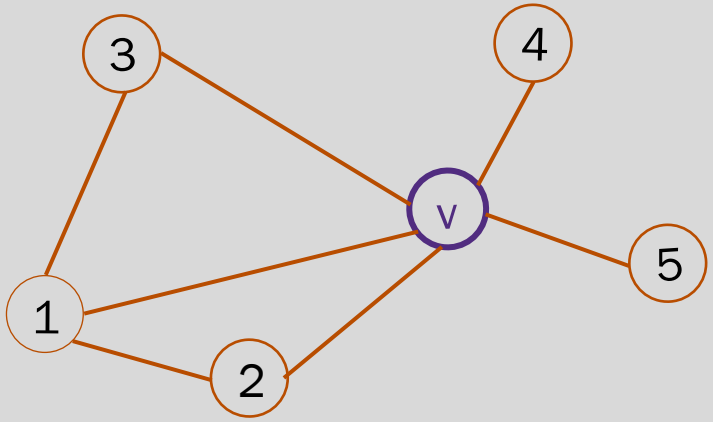
What is a random walk policy?

- As of yet, we haven't specified the random walk policy R
- Random walk policy definition
  - Assume the random walk is at node *v*
  - *Set a rule (policy) for deciding which neighbor node of v to select for the next step*
  - The rule can depend on parameters and other information about the walk state and network



How do we pick the random walk policy?

- Simplest approach is to select the next node in the walk from the neighbors of the current node with equal probability
  - Unbiased random walk
  - This was proposed for DeepWalk by Perozzi et al.
  - This turns out to be too restrictive
- How can we improve this?



Unbiased Random Walk Policy Example

Node *v* has 5 neighbors. Pick a number, ρ, from a uniform random distribution on [0,1]. Pick the next node, u, with the rule:

$\rho \in [0, 0.2) \longrightarrow u = 1$
$\rho \in (0.2, 0.4) \longrightarrow u = 2$
$\rho \in (0.4\ 0.6) \longrightarrow u = 3$
$\rho \in (0.6, 0.8) \longrightarrow u = 4$
$\rho \in (0.8, 1] \longrightarrow u = 5$

Breadth and depth first search

- **Breadth-first** is a walk policy that favors a local neighborhood view
- **Depth-first** is a walk policy that favors a global neighborhood views
- We would like a walk policy that combines these
- Next, we discuss *node2vec* which uses a parameterized bias random walk policy that interpolates between **breadth-first (local)** and **depth-first (global)** views of a node neighborhood

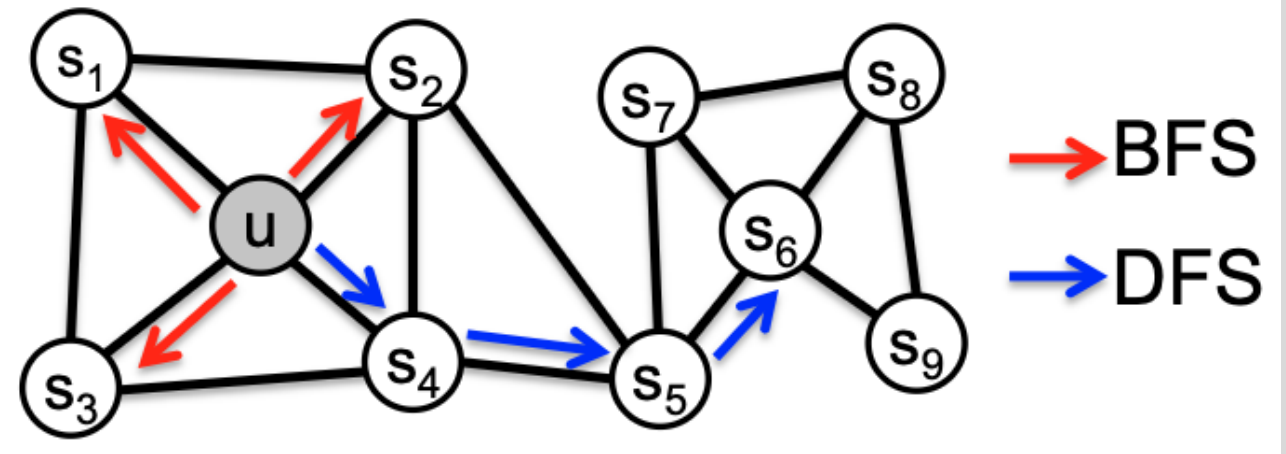
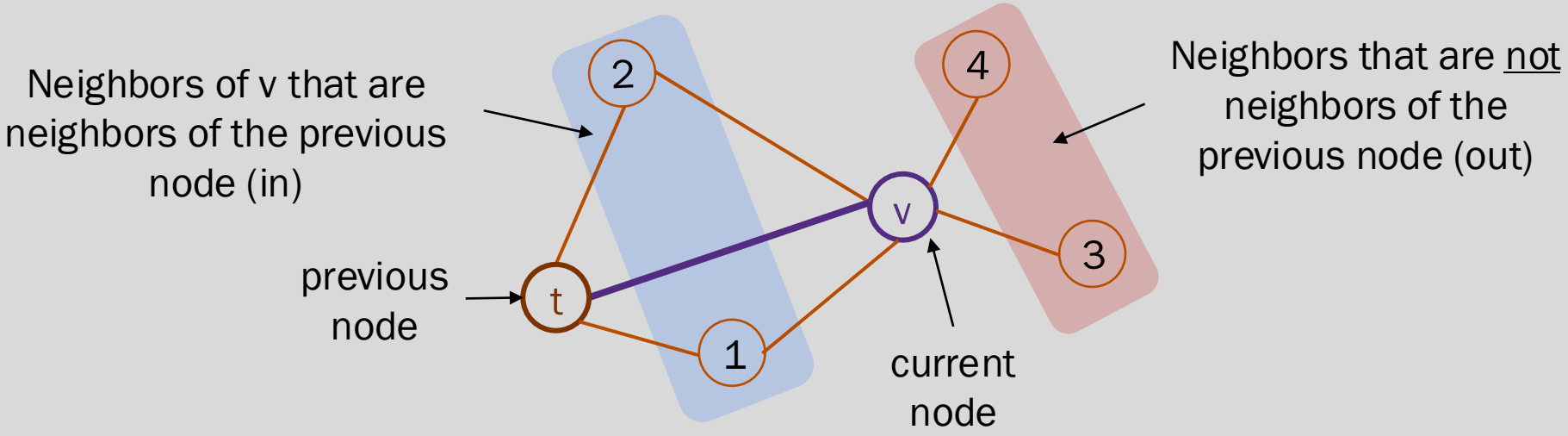


Image credit: Grover & Leskovec 2016

node2vec uses a biased random walk policy

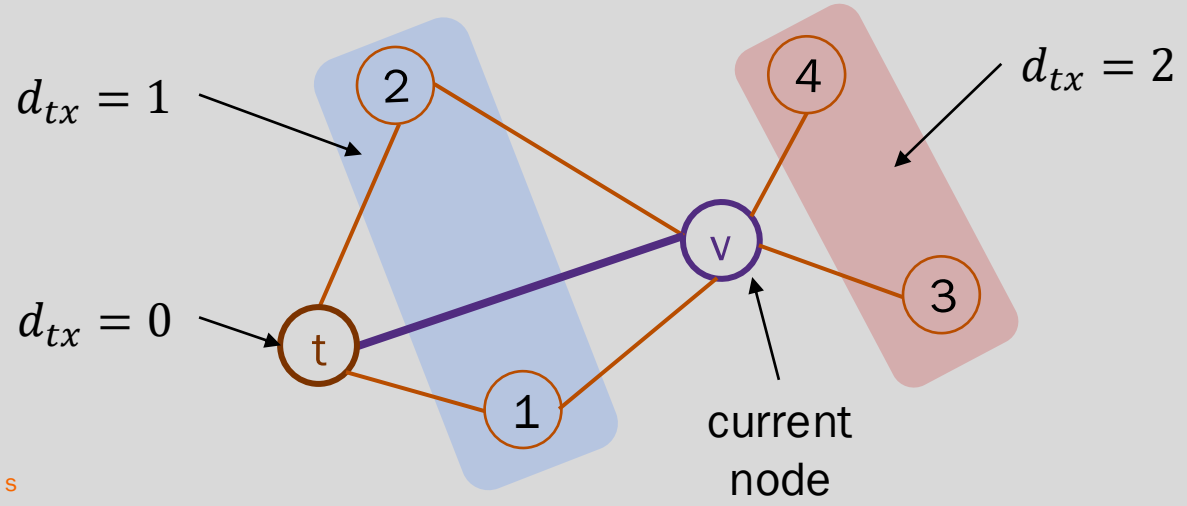
- Biased random walk concept
  - Assume the random walk just traversed the edge from node *t* to node *v*
  - **Bias the probability of picking a neighbor of node v based on whether it is**
    - The previous node
    - A neighbor of the previous node
    - Not a neighbor of the previous node
  - How do we bias the probability?



node2vec uses a biased random walk policy

- Biased probabilities
  - Assume the random walk just traversed the edge from node t to node v
  - Select policy parameters p and q (more on these later)
  - For node x in set of neighbors of v
    - Let w_{vx} be the weight on the edge between nodes v and x (unweighted $w_{vx} = 1$)
    - Let d_{tx} be the shortest path distance from node t to node x (d_{tx} is 0, 1, or 2)
    - Set the unnormalized transition probability to $\pi_{vx} = \alpha_{pq}(t,x) \cdot w_{vx}$
  - Normalize the transition probabilities by $\sum_{x \in N(v)} \pi_{vx}$

$$\alpha_{pq}(t,x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases}$$



node2vec biased random walk step example

- Let $\Pi_v = \sum_{x \in N(v)} \pi_{vx}$ be the sum of the unnormalized transition probabilities $\pi_{vx} = \alpha_{pq}(t,x) \cdot w_{vx}$ so that the normalized transition probability from node v to x is $\psi_{vx} = {}^{\pi_{vx}}/_{\Pi_v}$.

Example

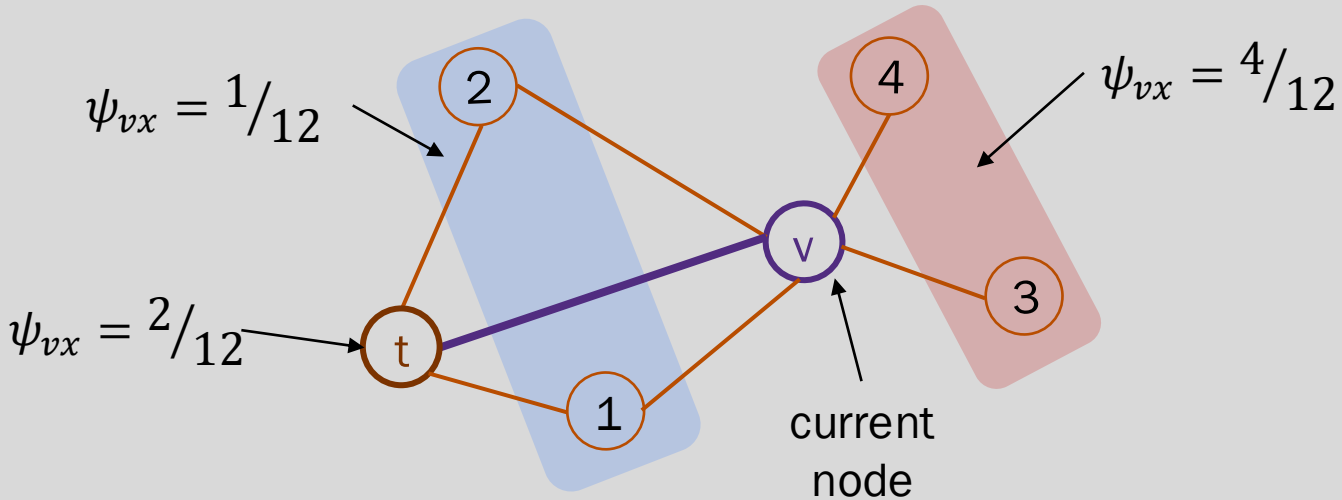
Let $w_{vx} = 1$ for all x (unweighted)

Let $p = 0.5$ and $q = 0.25$

$\pi_{vt} = 2, \pi_{v1} = \pi_{v2} = 1, \pi_{v3} = \pi_{v4} = 4$

$\Pi_v = \sum_{x \in N(v)} \pi_{vx} = 2 + 2 \cdot 1 + 2 \cdot 4 = 12$

$$\alpha_{pq}(t,x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases}$$



Pick a number, ρ, from a uniform random distribution on [0,1]. Pick the next node, u, with the rule:

$\rho \in [0, {}^{1}/_{12}) \longrightarrow u = 1$

$\rho \in ({}^{1}/_{12}, {}^{2}/_{12}) \longrightarrow u = 2$

$\rho \in ({}^{2}/_{12}, {}^{4}/_{12}) \longrightarrow u = t$

$\rho \in ({}^{4}/_{12}, {}^{8}/_{12}) \longrightarrow u = 3$

$\rho \in ({}^{8}/_{12}, 1] \longrightarrow u = 4$

https://ml4g.masinolab.com/topics/shallow-embeddings/node2vec.html

node2vec uses a biased random walk policy

- Random walk policy parameters
  - In-out parameter q: scales the probability of staying in (BFS) the previous node's neighbors vs. moving out (DFS) of the previous node's neighbors (DFS)
  - Setting $q > 1$ encourages BFS and $q < 1$ encourages DFS (we saw this in the previous example)
  - Return parameter p: scales the probability of returning to the previous node in a walk
  - Setting $p > \max(q, 1)$ discourages backtracking, setting $p < min(q, 1)$ encourages backtracking, otherwise $min(q, 1) < p < \max(q, 1)$ has moderate backtracking

https://ml4g.masinolab.com/topics/shallow-embeddings/node2vec.html

node2vec context sampling

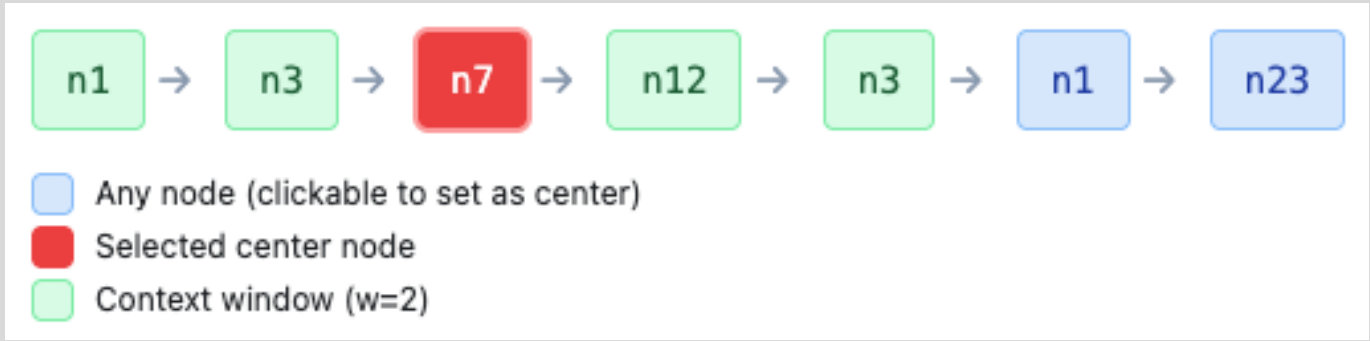
- Assume we have created a large number of random walks with the node2vec biased walk policy
- How do we use them to create samples?
- Consider a single random walk

1. Set the context window length w
2. Pick a center node, u, in the walk
3. Let every node, v, that is within w steps to u be a context node
4. Form positive pairs (u, v) for u and every context node
5. Form a single sample as the set of positive pairs and k randomly sampled negative nodes (i.e., those not in the graph that are not in the context nodes)

https://ml4g.masinolab.com/topics/shallow-embeddings/node2vec.html

node2vec context sampling

A single random walk of length 6

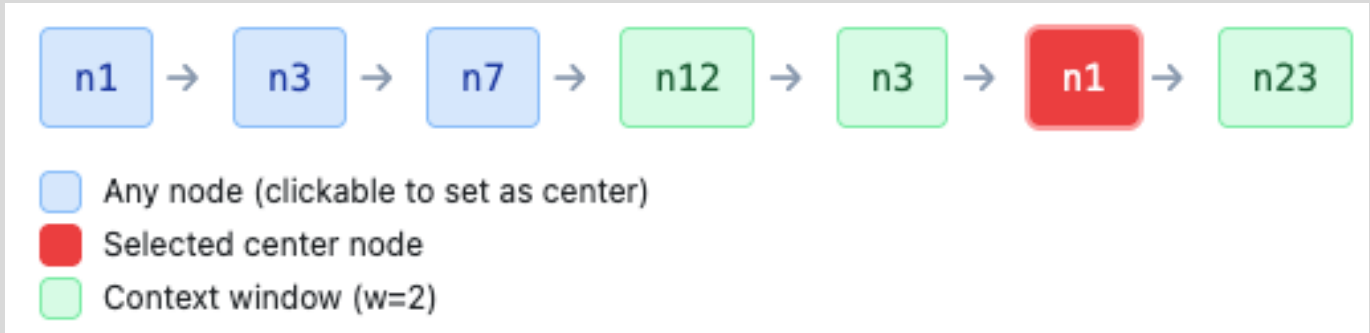


Context Window for center node n7 at position 2

Context nodes: `n1, n3, n12, n3`

Window range: `[0, 4]`

Positive pairs:
`(n7, n1), (n7, n3), (n7, n12), (n7, n3)`



Context Window for center node n1 at position 5

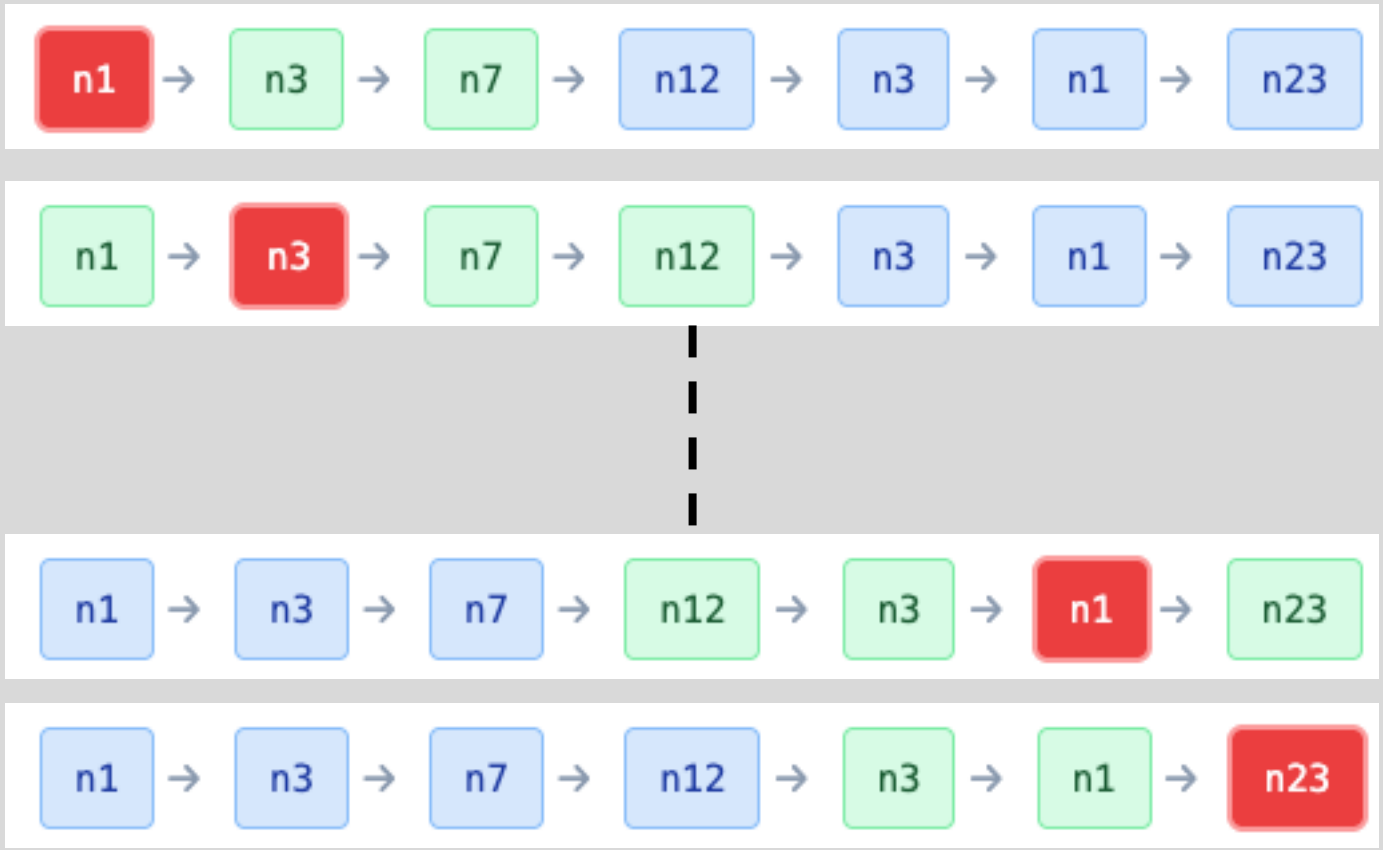
Context nodes: `n12, n3, n23`

Window range: `[3, 6]`

Positive pairs:
`(n1, n12), (n1, n3), (n1, n23)`

https://ml4g.masinolab.com/topics/shallow-embeddings/node2vec.html

node2vec context sampling



Each row is a single sample for SGD

Sample	Walk	Center Pos	Center Node	Context	Positive Pairs	Negative Samples
1	1	0	n1	n3, n7	(n1, n3), (n1, n7)	n45, n29, n86, n63, n24
2	1	1	n3	n1, n7, n12	(n3, n1), (n3, n7), (n3, n12)	n41, n86, n99, n72, n27
3	1	2	n7	n1, n3, n12, n3	(n7, n1), (n7, n3), (n7, n12), (n7, n3)	n14, n5, n81, n61, n66
4	1	3	n12	n3, n7, n3, n1	(n12, n3), (n12, n7), (n12, n3), (n12, n1)	n44, n68, n15, n81, n37
5	1	4	n3	n7, n12, n1, n23	(n3, n7), (n3, n12), (n3, n1), (n3, n23)	n74, n15, n28, n20, n19
6	1	5	n1	n12, n3, n23	(n1, n12), (n1, n3), (n1, n23)	n81, n2, n54, n87, n26
7	1	6	n23	n3, n1	(n23, n3), (n23, n1)	n83, n29, n27, n45, n87

https://ml4g.masinolab.com/topics/shallow-embeddings/node2vec.html

node2vec algorithm summary

1. Precompute edge transition probabilities for all edges
2. Generate r random walks of length l for every node in the graph using the bias random walk policy
3. Use context sampling to generate samples where each sample include positive pairs (u, v_w) and k negative nodes
4. Apply SGD over each sample using NCE with negative sampling loss

- Linear time complexity
- All steps are individually parallelizable

https://ml4g.masinolab.com/topics/shallow-embeddings/node2vec.html

Random walk variants

- Different kinds of biased random walks:
  - Based on node attributes (Dong et al.)
  - Based on learned weights (Abu-El-Haija et al.)
- Alternative optimization schemes:
  - Directly optimize based on 1-hop and 2-hop random walk probabilities (as in LINE from Tang et al.)
- Network preprocessing techniques:
  - Run random walks on modified versions of the original network (e.g., Ribeiro et al. struct2vec, Chen et al. HARP)

Summary so far

- Core idea: Embed nodes so that distances in embedding space reflect node similarities in the original network
- **Different notions of node similarity:**
  - Naïve: Similar if two nodes are connected
  - Random walk approaches

Summary so far

- So, what method should I use..?
- **No one method wins in all cases....**
  - E.g., node2vec performs better on node classification while alternative methods perform better on link prediction (Goyal and Ferrara, survey)
- Random walk approaches are generally more efficient
- In general: Must choose definition of node similarity that matches your application

School of COMPUTING
Clemson University

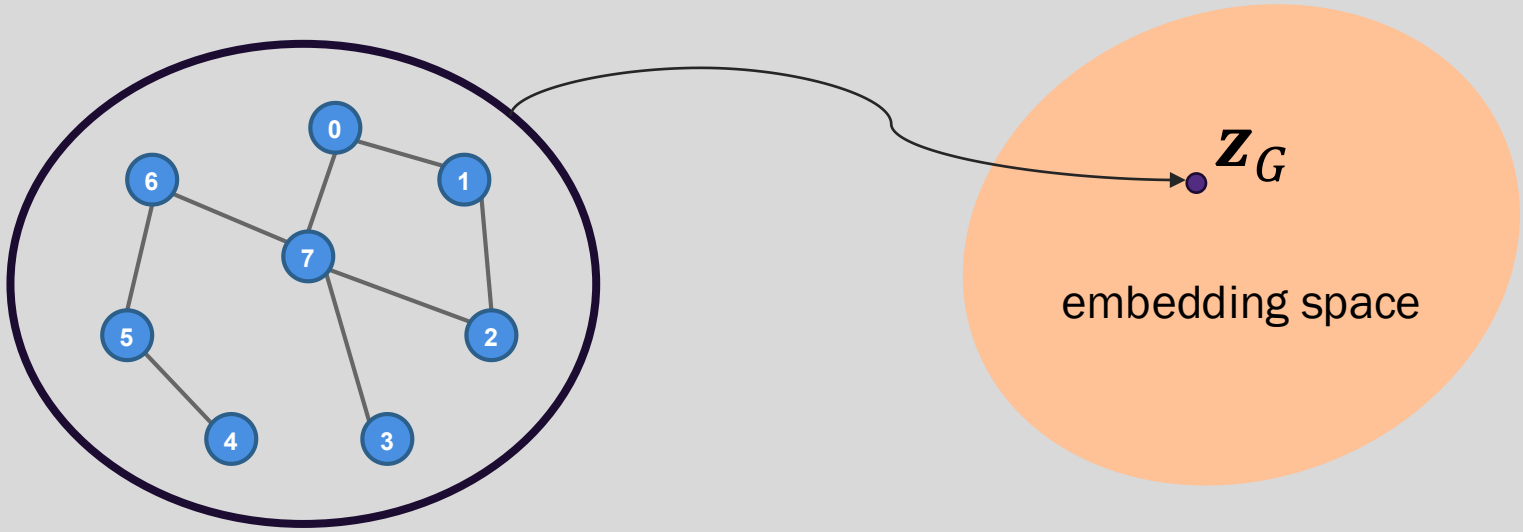
Graph Embeddings

Embedding an entire graph

Goal: Embed a subgraph (or the entire graph) G in the embedding space

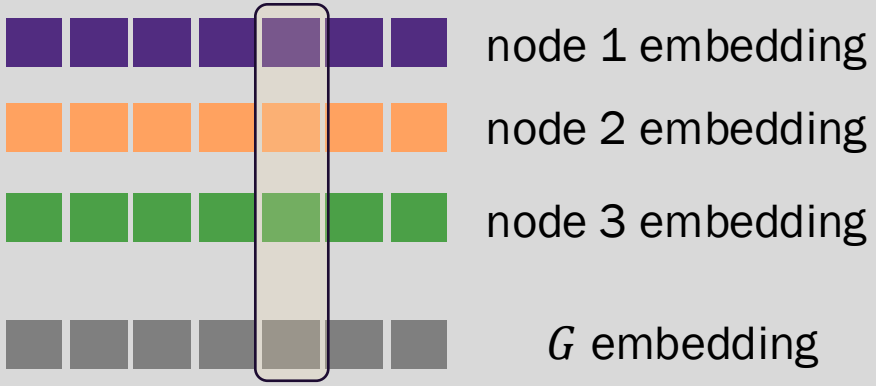
Motivation: Classify or cluster networks

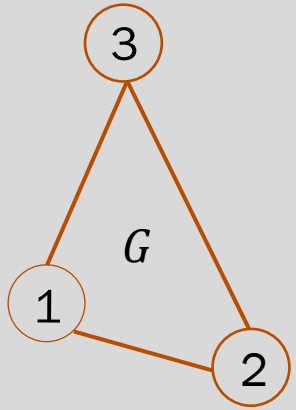
Examples: candidate drug targets, anomalous networks



Node averaging for graph embedding

- Apply a node embedding algorithm to all nodes in G
- Average the node embeddings to obtain a single embedding for G
- The average is done element wise

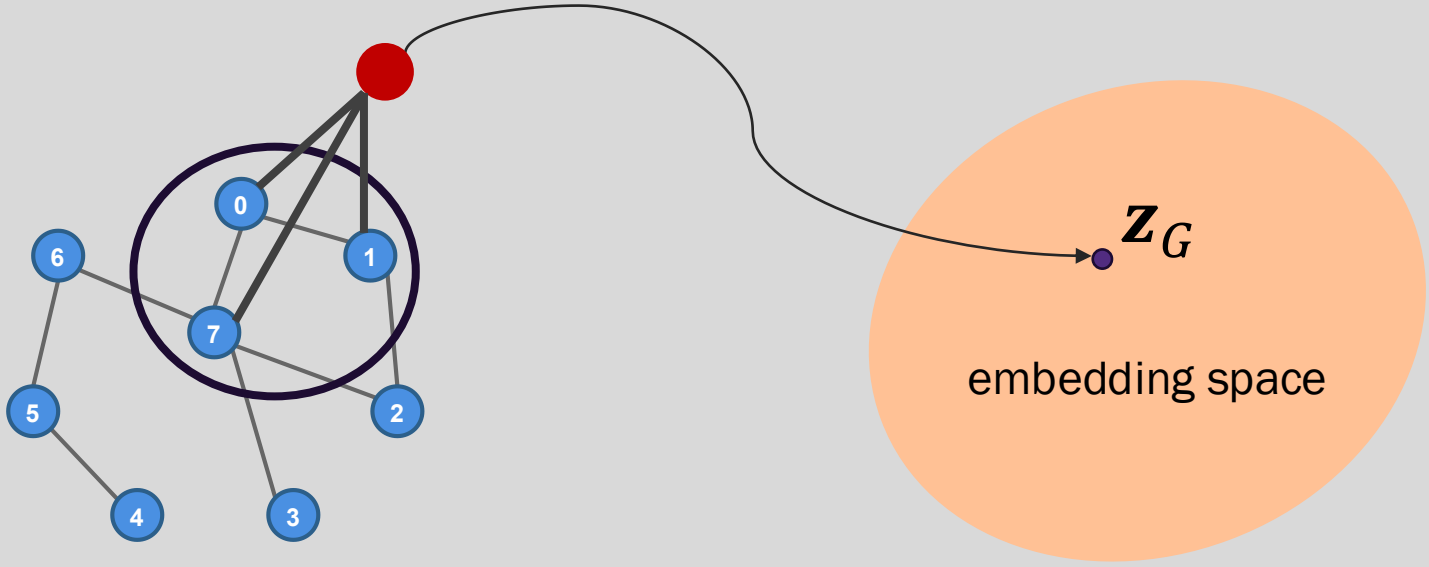




position *i* in G embedding is the average of the embedding values at position *i* from the node embeddings

Virtual nodes for graph embedding

- Create a virtual node and connect all nodes in the subgraph (or entire graph) to the virtual node
- Run a node embedding method to obtain the embedding for the virtual node



School of COMPUTING
Clemson University

Limitations of matrix factorization and random walk embeddings

Matrix factorization and random walk embedding methods are transductive

- Transductive (not inductive) method:
  - Cannot obtain embeddings for nodes not in the training set
  - Cannot apply to new graphs using the same nodes or the same graph with updated edges
    - If you apply a random walk method to the same graph multiple times, you'll get a different embedding each time. Why?

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Random walk embedding methods cannot capture structural similarity

- DeepWalk and node2vec do not capture structural similarity
- Consider the network below
  - Node 1 and 11 are structurally similar – part of one triangle, degree 2, …
  - However, they have very different random walk embeddings.
    - Why? It's unlikely that a random walk will reach node 11 from node 1

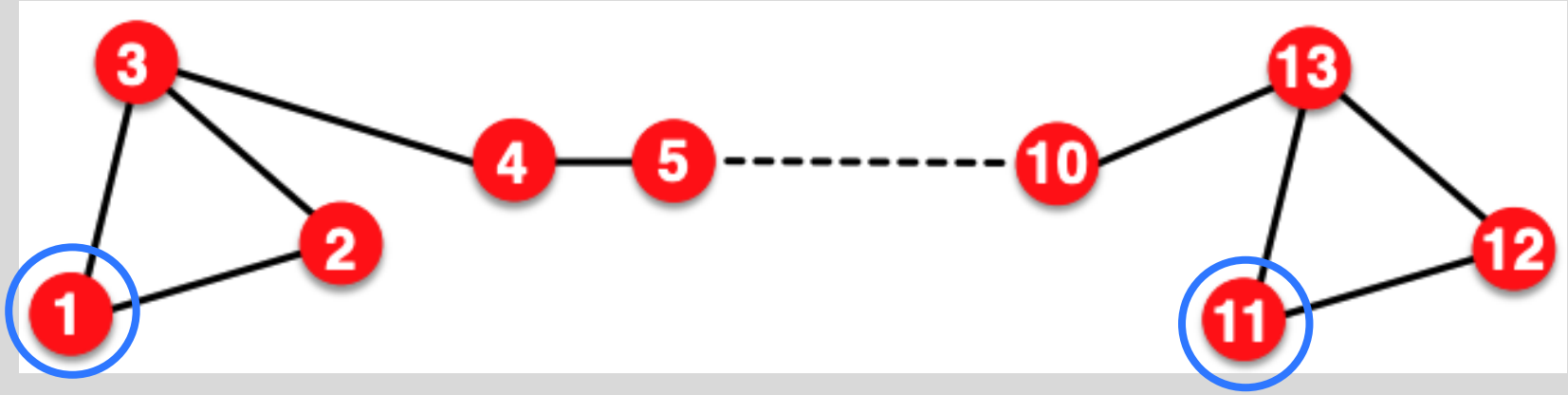


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Matrix factorization and random walk embedding methods cannot use node, edge, or graph features

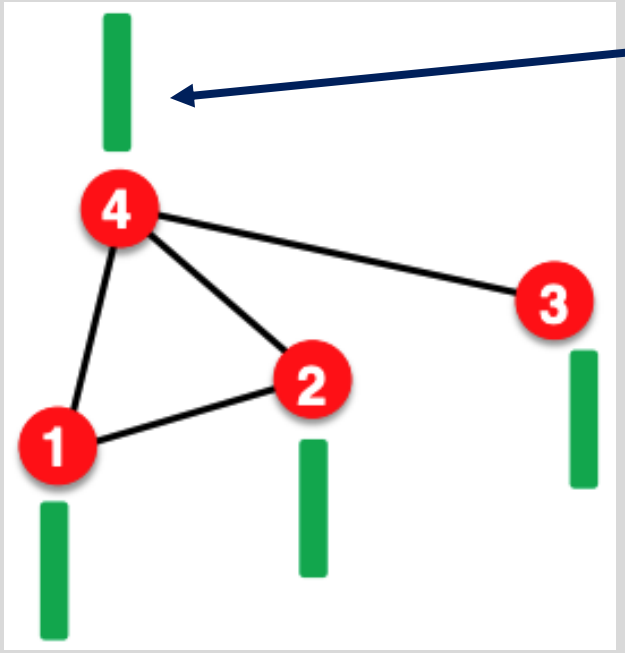


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Node feature vector
(e.g. protein properties in a protein-protein interaction graph)

Matrix factorization / random walk embeddings do not incorporate such node features

Solution: Graph neural networks

What can we do with node embeddings?

- Clustering/community detection: Cluster points $\mathbf{z}_n$
- Node classification: Predict label of node n based on $\mathbf{z}_n$
- Link prediction: Predict edge (u, v) based on a function of $f\big(g(\mathbf{z}_u, \mathbf{z}_v)\big)$ where the function g is usually on of vector concatenation, averaging, Hadamard product, or difference
- Graph classification: Predict label based on graph embedding, $\mathbf{z}_G$, formed via aggregating node embeddings or virtual-node